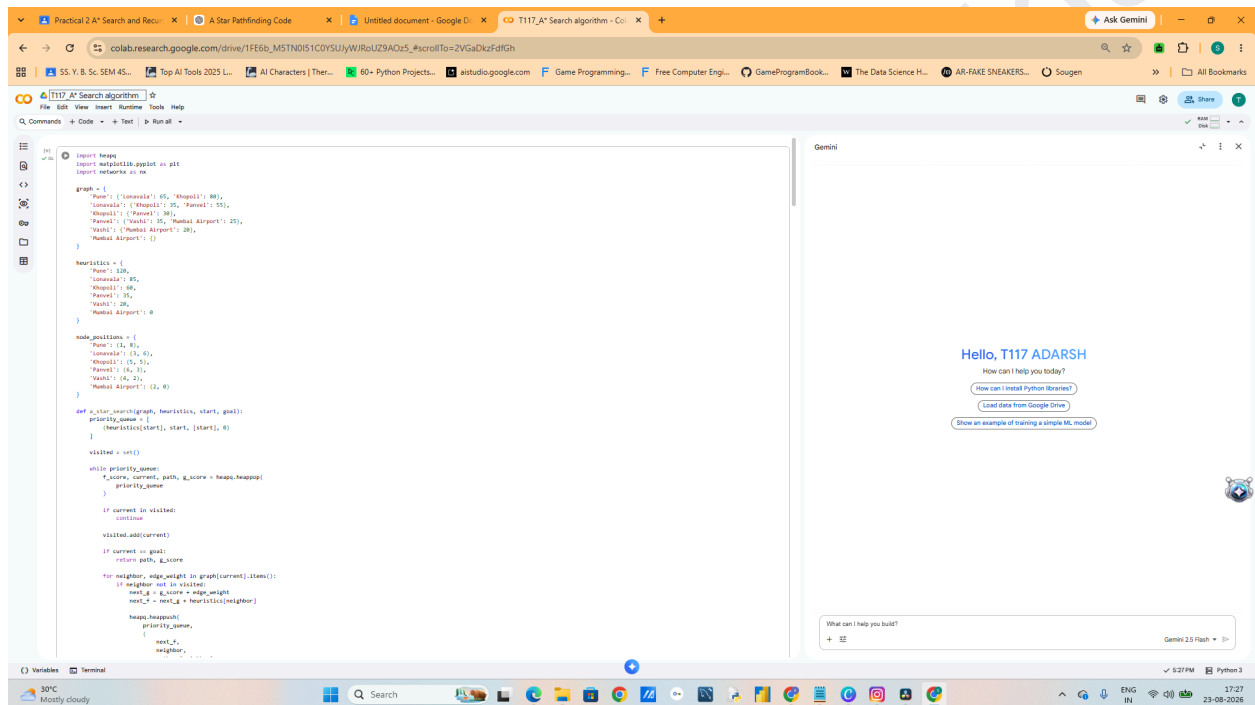


Practical 1 A* Search and Recursive Best-First Search

AIM A* Search and Recursive Best-First Search

Code :



The screenshot displays a Google Colab environment. On the left, a Jupyter notebook titled 'T117 A* Search algorithm' contains Python code for an A* search algorithm. The code defines a graph with nodes like 'Pune', 'Mumbai', 'Mumbai Airport', and 'Mumbai Airport'. It includes a heuristic function and a search function that uses a priority queue to find the shortest path. On the right, the Gemini chat interface is visible, showing a greeting 'Hello, T117 ADARSH' and several buttons for user actions like 'How can I help you today?', 'How can I install Python libraries?', 'Load data from Google Drive', and 'Show an example of training a simple ML model'. The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 17:27 on 23-08-2028.

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

graph = nx.Graph()
graph.add_weighted_edges_from([
    ('Pune', 'Mumbai', 40, 'Road'),
    ('Mumbai', 'Mumbai Airport', 20, 'Road'),
    ('Mumbai', 'Pune', 40, 'Road'),
    ('Mumbai', 'Mumbai Airport', 20, 'Road'),
    ('Mumbai Airport', 'Mumbai', 20, 'Road')
])

heuristic = {
    'Pune': 10,
    'Mumbai': 0,
    'Mumbai Airport': 0
}

start_pos = ('Pune', 0)
goal_pos = ('Mumbai Airport', 0)

def a_star_search(graph, heuristic, start, goal):
    priority_queue = [(heuristic[start], start, [start], 0)]
    visited = set()

    while priority_queue:
        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

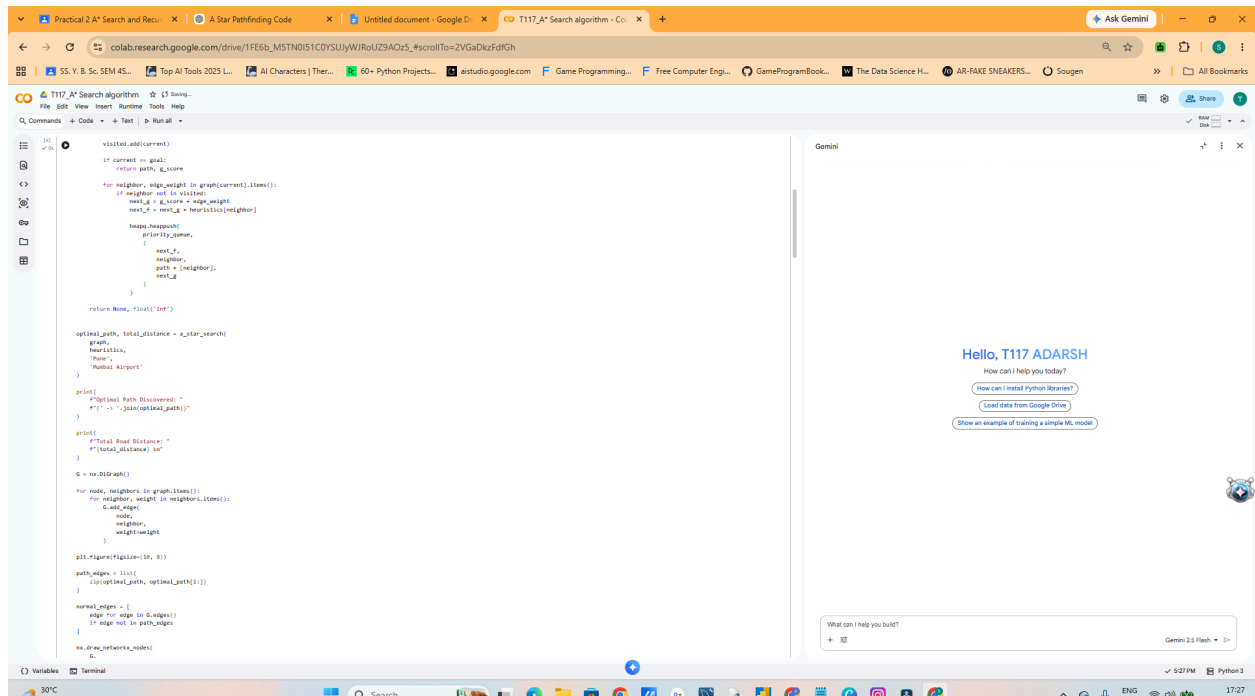
        if current == goal:
            return path, g_score

        for neighbor, edge_weight in graph[current].items():
            if neighbor not in visited:
                next_g = g_score + edge_weight
                next_f = next_g + heuristic[neighbor]
                heapq.heappush(priority_queue, (next_f, neighbor, path + [neighbor], next_g))
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Practical 1 A* Search and Recursive Best-First Search



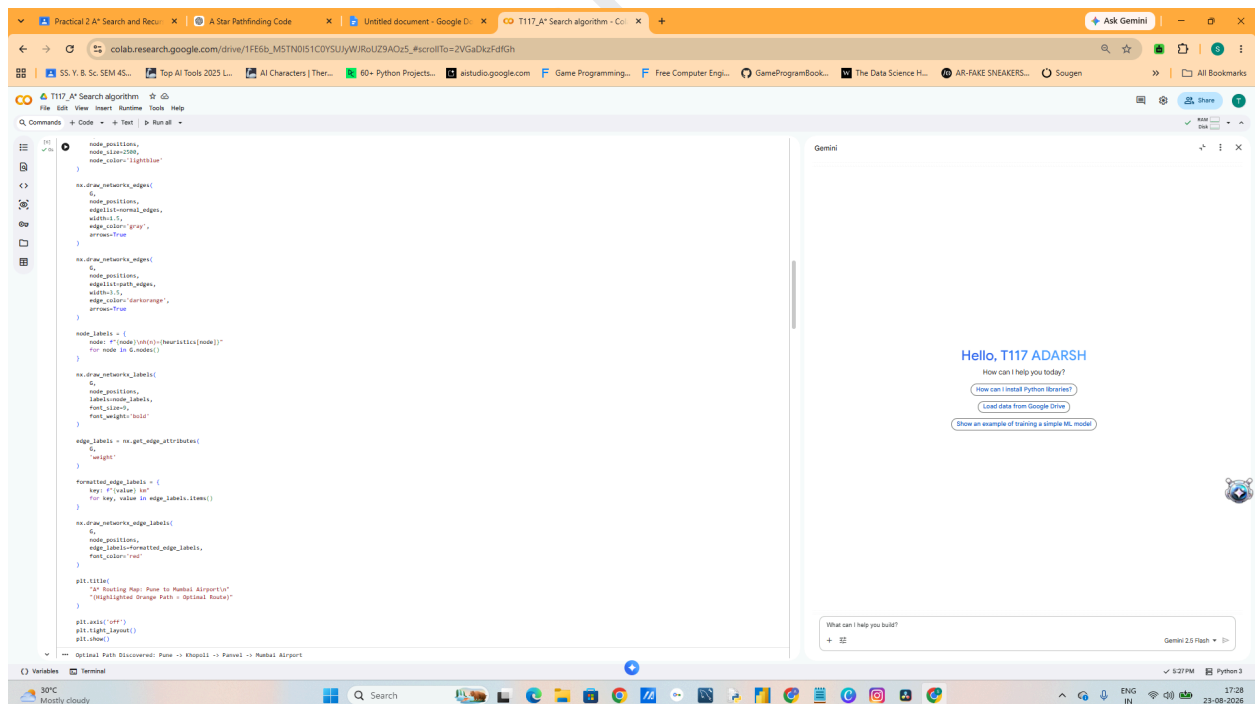
The screenshot shows a Google Colab notebook titled "T117_A* Search algorithm". The code implements an A* search algorithm to find the shortest path from "Pune" to "Mumbai Airport". The code includes functions for calculating the heuristic (Euclidean distance), adding nodes to a priority queue, and returning the optimal path and distance. The output of the code is printed to the console.

```
[1] In [ ]: visited.add(current)
          if current == goal:
              return path, g_score
          for neighbor, edge_weight in graph[current].items():
              if neighbor not in visited:
                  new_g = g_score + edge_weight
                  new_f = new_g + heuristic(neighbor)
                  heapq.heappush(
                      priority_queue,
                      (new_f,
                       neighbor,
                       path + [neighbor],
                       new_g)
                  )
          return None, float('inf')

def optimal_path, total_distance = a_star_search(
    graph,
    heuristic,
    "Pune",
    "Mumbai Airport"
)

print(f"Optimal Path Discovered: {optimal_path}")
print(f"Total Road Distance: {total_distance} km")
G = nx.DiGraph()
for node, neighbors in graph.items():
    for neighbor, weight in neighbors.items():
        G.add_edge(
            node,
            neighbor,
            weight=weight
        )
plt.figure(figsize=(20, 8))
path_edges = []
for i in range(len(optimal_path)-1):
    path_edges.append((optimal_path[i], optimal_path[i+1]))
normal_edges = []
for edge in G.edges():
    if edge not in path_edges:
        normal_edges.append(edge)
nx.draw_networkx_nodes(G,
                        node_pos={},
                        node_size=200,
                        node_color="lightblue")
nx.draw_networkx_edges(G,
                        node_pos={},
                        edge_pos={},
                        width=1,
                        edge_color="gray",
                        arrows=True)
node_labels = {}
for node in G.nodes():
    node_labels[node] = heuristic(node)
nx.draw_networkx_labels(G,
                        node_pos={},
                        label=node_labels,
                        font_size=10,
                        font_weight="bold")
edge_labels = nx.get_edge_attributes(G,
                                     "weight")
formatted_edge_labels = {}
for key, value in edge_labels.items():
    formatted_edge_labels[key] = f"{value} km"
nx.draw_networkx_labels(G,
                        node_pos={},
                        edge_labels=formatted_edge_labels,
                        font_size=10)
plt.title("A* Routing Map: Pune to Mumbai Airport")
plt.show()
```

The Gemini chat interface on the right shows a greeting: "Hello, T117 ADARSH".



The screenshot shows a Google Colab notebook titled "T117_A* Search algorithm". The code implements an A* search algorithm to find the shortest path from "Pune" to "Mumbai Airport". The code includes functions for calculating the heuristic (Euclidean distance), adding nodes to a priority queue, and returning the optimal path and distance. The output of the code is printed to the console.

```
[1] In [ ]: node_pos={},
          node_size=200,
          node_color="lightblue"
          nx.draw_networkx_edges(G,
                                node_pos={},
                                edge_pos={},
                                width=1,
                                edge_color="gray",
                                arrows=True)
          node_labels = {}
          for node in G.nodes():
              node_labels[node] = heuristic(node)
          nx.draw_networkx_labels(G,
                                node_pos={},
                                label=node_labels,
                                font_size=10,
                                font_weight="bold")
          edge_labels = nx.get_edge_attributes(G,
                                               "weight")
          formatted_edge_labels = {}
          for key, value in edge_labels.items():
              formatted_edge_labels[key] = f"{value} km"
          nx.draw_networkx_labels(G,
                                node_pos={},
                                edge_labels=formatted_edge_labels,
                                font_size=10)
          plt.title("A* Routing Map: Pune to Mumbai Airport")
          plt.show()
          print(f"Optimal Path Discovered: Pune -> Khopoli -> Parvel -> Mumbai Airport")
```

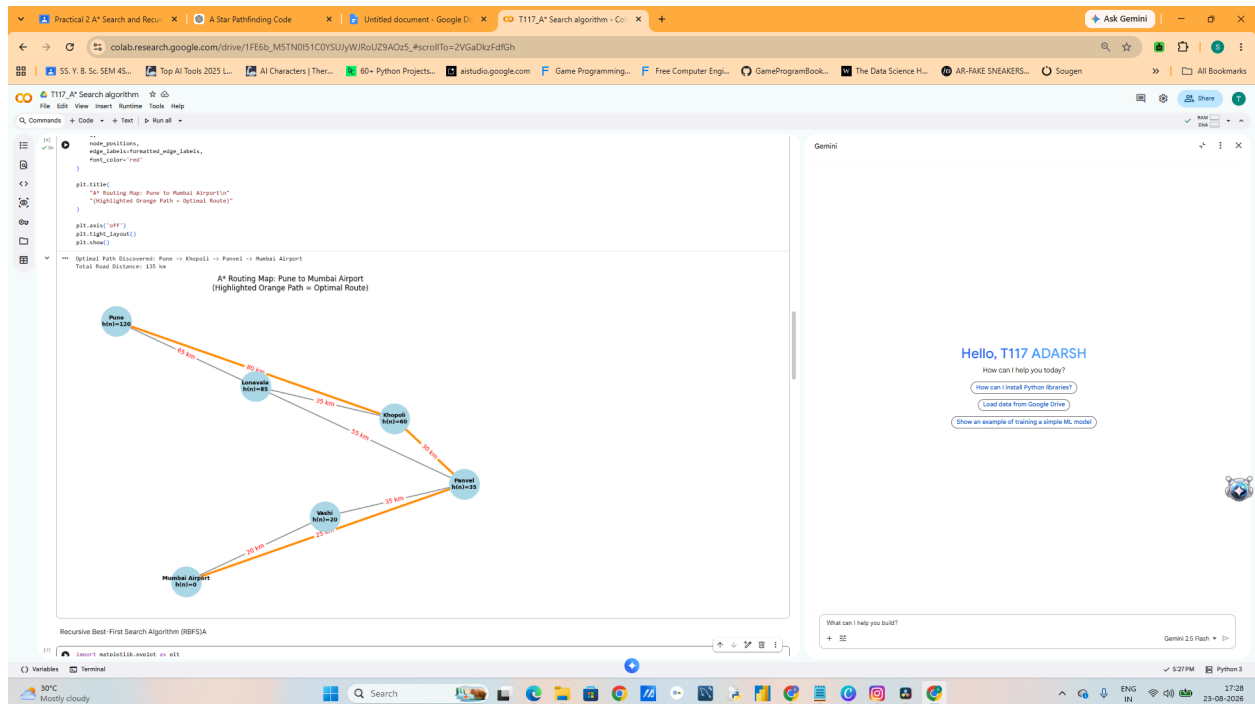
The Gemini chat interface on the right shows a greeting: "Hello, T117 ADARSH".

Output :

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Practical 1 A* Search and Recursive Best-First Search



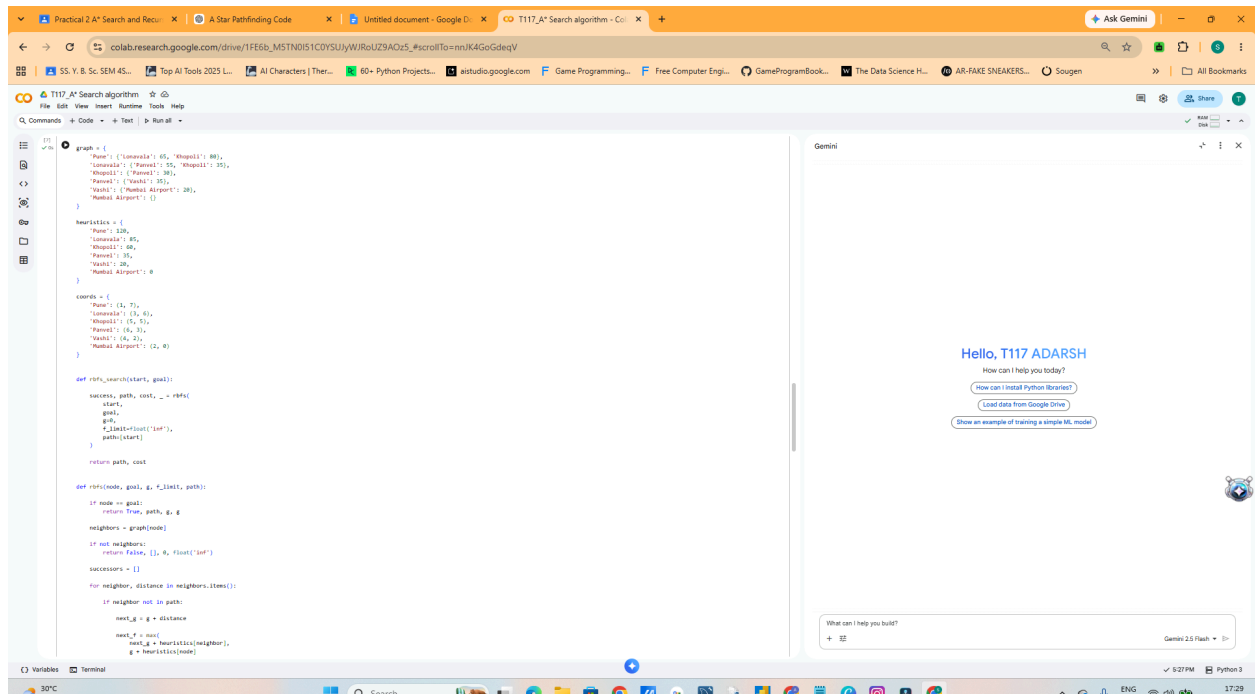
Recursive Best-First Search Algorithm (RBFS)A

Code :

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Practical 1 A* Search and Recursive Best-First Search



```
graph = {
    "Pune": ("Lonavala": 45, "Khopoli": 60),
    "Lonavala": ("Pune": 45, "Khopoli": 35),
    "Khopoli": ("Pune": 35),
    "Pune": ("Mumbai": 30),
    "Mumbai": ("Mumbai Airport": 20),
    "Mumbai Airport": {}
}

heuristic = {
    "Pune": 120,
    "Lonavala": 85,
    "Khopoli": 40,
    "Pune": 35,
    "Mumbai": 30,
    "Mumbai Airport": 0
}

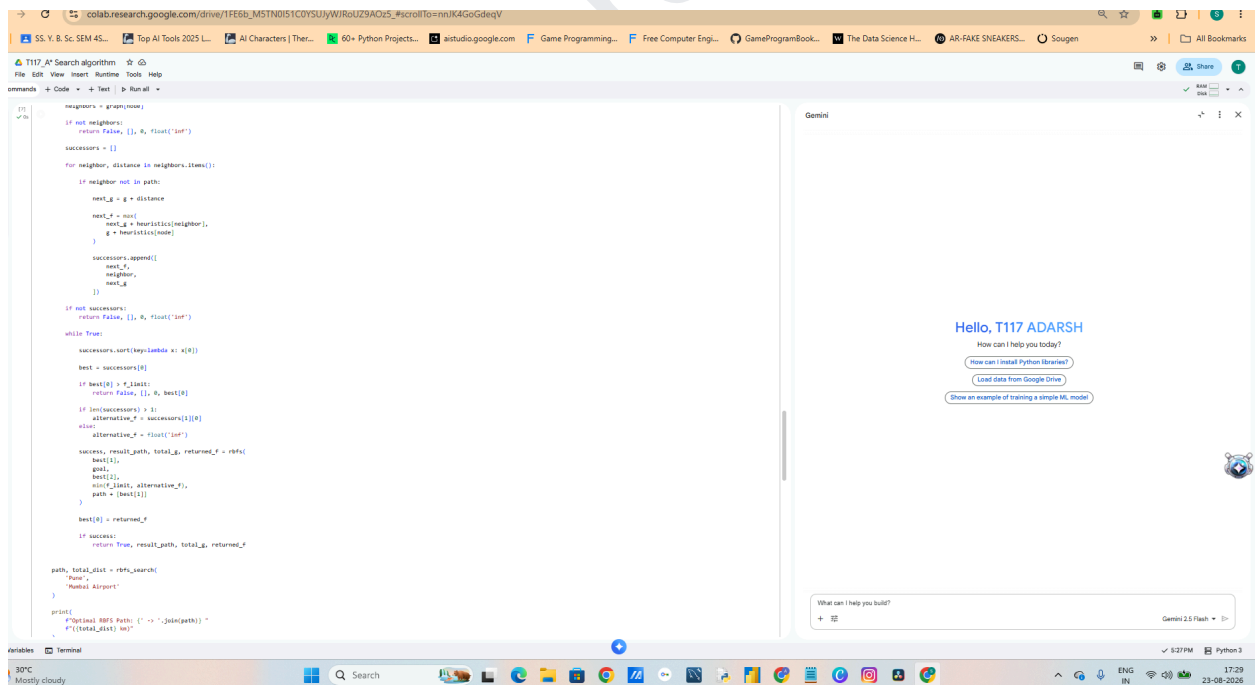
coords = {
    "Pune": (2, 7),
    "Lonavala": (3, 4),
    "Khopoli": (5, 5),
    "Pune": (6, 2),
    "Mumbai": (4, 2),
    "Mumbai Airport": (2, 4)
}

def rbf_search(start, goal):
    successors, path, cost, _ = RBF(
        start,
        goal,
        f_heuristic=heuristic,
        path_cost=cost
    )
    return path, cost

def rbf(node, goal, g, f_heuristic, path):
    if node == goal:
        return True, path, g, g
    neighbors = graph[node]
    if not neighbors:
        return False, [], 0, float('inf')
    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            new_g = g + distance
            new_f = cost
            new_g = heuristic[neighbor],
            g = heuristic[node]
            successors.append([
                new_f,
                neighbor,
                new_g
            ])
    if not successors:
        return False, [], 0, float('inf')
    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]
        if best[0] > f_heuristic:
            return False, [], 0, best[0]
        if len(successors) > 1:
            alternative_f = successors[1][0]
            if alternative_f < f_heuristic:
                successors, result_path, total_g, returned_f = rbf(
                    best[1],
                    goal,
                    best[2],
                    alternative_f,
                    path = [best[1]]
                )
                best[0] = returned_f
        if best[0] < returned_f:
            return True, result_path, total_g, returned_f

path, total_cost = rbf_search(
    "Pune",
    "Mumbai Airport"
)

print(
    f"Optimal RBF Path: ' -> '.join(path)"
    f"Cost: {total_cost}"
)
```



```
neighbors = graph[node]
if not neighbors:
    return False, [], 0, float('inf')

successors = []

for neighbor, distance in neighbors.items():
    if neighbor not in path:
        new_g = g + distance
        new_f = cost
        new_g = heuristic[neighbor],
        g = heuristic[node]
        successors.append([
            new_f,
            neighbor,
            new_g
        ])
    if not successors:
        return False, [], 0, float('inf')
    while True:
        successors.sort(key=lambda x: x[0])
        best = successors[0]
        if best[0] > f_heuristic:
            return False, [], 0, best[0]
        if len(successors) > 1:
            alternative_f = successors[1][0]
            if alternative_f < f_heuristic:
                successors, result_path, total_g, returned_f = rbf(
                    best[1],
                    goal,
                    best[2],
                    alternative_f,
                    path = [best[1]]
                )
                best[0] = returned_f
        if best[0] < returned_f:
            return True, result_path, total_g, returned_f

path, total_cost = rbf_search(
    "Pune",
    "Mumbai Airport"
)

print(
    f"Optimal RBFS Path: ' -> '.join(path)"
    f"Cost: {total_cost}"
)
```

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Practical 1 A* Search and Recursive Best-First Search

The image displays two screenshots of a Google Colab environment. The top screenshot shows a Python script for an A* search algorithm. The script defines a graph with nodes and edges, and implements the A* search algorithm to find the shortest path from Pune to Mumbai Airport. The script uses a priority queue to explore nodes and a heuristic function to estimate the distance to the goal. The output of the script is displayed in the terminal, showing the shortest path and the total cost.

```
graph LR
    Pune --> Khepail
    Pune --> Farvel
    Pune --> Vashi
    Pune --> Mumbai_Airport
    Khepail --> Farvel
    Khepail --> Vashi
    Khepail --> Mumbai_Airport
    Farvel --> Vashi
    Farvel --> Mumbai_Airport
    Vashi --> Mumbai_Airport
```

The bottom screenshot shows the same Colab environment, but with the script modified to include a recursive best-first search algorithm. The script defines a graph with nodes and edges, and implements the recursive best-first search algorithm to find the shortest path from Pune to Mumbai Airport. The script uses a priority queue to explore nodes and a heuristic function to estimate the distance to the goal. The output of the script is displayed in the terminal, showing the shortest path and the total cost.

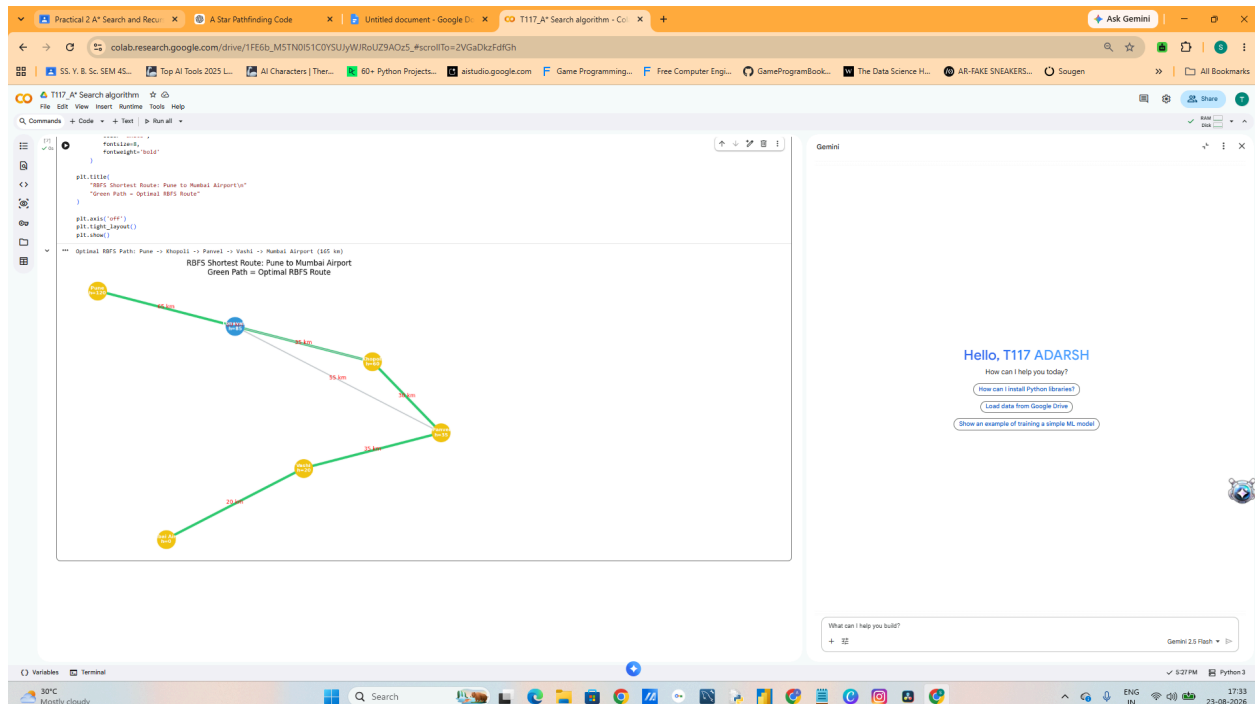
```
graph LR
    Pune --> Khepail
    Pune --> Farvel
    Pune --> Vashi
    Pune --> Mumbai_Airport
    Khepail --> Farvel
    Khepail --> Vashi
    Khepail --> Mumbai_Airport
    Farvel --> Vashi
    Farvel --> Mumbai_Airport
    Vashi --> Mumbai_Airport
```

Output :

Name : adarsh shukla
Roll no : T117

SHETH L.U.J AND SIR M.V. COLLEGE
SUBJECT NAME: Artificial Intelligence

Practical 1 A* Search and Recursive Best-First Search



Name : adarsh shukla
Roll no : T117